Side-effects — Amaliy mashg'ulot

LaunchedEffect · rememberCoroutineScope · rememberUpdatedState · DisposableEffect · SideEffect · produceState · derivedStateOf · snapshotFlow (13-Ma'ruza)

Talaba: Sirojbek Xolmatov | Guruh: AK2024010 | Sana: 30.10.2026

> *Mashg'ulot ikki qismdan iborat: A — nazariy savollar va kod bilan ishlash, B — Android Studio'da bajariladigan amaliy topshiriqlar.*

A qism. Nazariy savollar va javoblar

1-savol

Kontekst: Dasturchi ProfileScreen composable tanasining boshida viewModel.loadProfile() va analytics.log("opened") ni chaqiradi. Ekranda animatsiya ishlaganda so'rovlar va log yozuvlari ko'payib ketdi.

Vazifa: Side-effect nima ekanini ta'riflang. Nega composable tanasi side-effect uchun xavfli joy: recomposition qanday sharoitda ko'p marta yoki bekor qilinib (discard) qayta boshlanishi mumkin? Ma'ruzadagi "effect" ta'rifini (Key Term) o'z so'zlaringiz bilan yozing va nega ma'lumot yuklash ViewModel'ga tegishli ekanini (unidirectional data flow) tushuntiring.

Javob:
Side-effect — bu composable funksiya doirasidan tashqarida sodir boladigan va ilova holatini ozgartiradigan har qanday amal (masalan bazaga yozish, serverdan malumot yuklash, analytics yuborish).
Composable tanasi side-effect uchun juda xavfli joy chunki Compose animatsiyalar yoki state ozgarishida funksiyani sekundiga 60 marta qayta chaqirishi (recomposition) yoki bekor qilib (discard) tashlashi mumkin. Tanada yozilgan kodlar tartibsiz va yuzlab marta keraksiz ishlaydi.
Key Term bo'yicha Effect: Bu UI ga xalaqit bermasdan, uning hayot siklini boshqarib turuvchi xavfsiz blok.
Unidirectional data flow (UDF): Malumot yuklash ViewModelning vazifasi hisoblanadi, UI faqat tayyor state ni olib chizib berishi kerak.

2-savol

Kontekst: Jamoa Compose'dagi sakkizta effect API'ni ajrata olmayapti va har bir vaziyatda LaunchedEffect ishlatmoqda.

Vazifa: Jadvalning o'ng ustunini to'ldiring: har bir ehtiyoj uchun qaysi API mos keladi?

Men xohlayman	API
Composable Composition'da turganda suspend funksiyani chaqirish	**LaunchedEffect**
onClick kabi callback ichidan coroutine ishga tushirish	**rememberCoroutineScope**
Effect ichida eng so'nggi qiymatni o'qish, lekin effect'ni qayta ishga tushirmaslik	**rememberUpdatedState**
Listener qo'shish va keyin uni olib tashlash	**DisposableEffect**
Compose state'ni Compose boshqarmaydigan kodga uzatish	**SideEffect**
Flow, LiveData yoki RxJava'ni Compose State'ga aylantirish	**produceState**
Kirish qiymatidan kamroq recompose qilish	**derivedStateOf**
Compose State'ni Flow'ga aylantirish	**snapshotFlow**

3-savol

Kontekst: Quyidagi composable ekran Composition'ga kirganda pulsatsiya animatsiyasini ishga tushiradi:

```
var pulseRateMs by remember { mutableLongStateOf(3000L) }
val alpha = remember { Animatable(1f) }
LaunchedEffect(pulseRateMs) {
    while (isActive) {
        delay(pulseRateMs)
        alpha.animateTo(0f)
        alpha.animateTo(1f)
    }
}
```

Vazifa: (1) Effect qachon ishga tushadi? (2) pulseRateMs 3000 dan 1000 ga o'zgarganda nima bo'ladi? (3) Composable Composition'ni tark etganda while siklining taqdiri qanday bo'ladi va isActive nimani tekshiradi? Ma'ruzadagi MiniComposition modeli (effect(100) cancelled) bu xatti-harakatni qanday ko'rsatadi?

> **Javob:**
> (1) Effect birinchi marta ekranga chiqqanda yoki pulseRateMs qiymati ozgarganda ishga tushadi.
> (2) pulseRateMs 3000 dan 1000 ga ozgarganda eski korutina bekor qilinadi (cancel) va yangi 1000 ms tezlik bilan qaytadan boshlanadi.
> (3) Composable composition'ni tark etganda korutina avtomat to'xtatiladi. isActive korutina bekor qilingan-qilinmaganini tekshiradi, agar to'xtatilgan bolsa while sikli darhol yakunlanadi.
> MiniComposition modelida bu holat `effect(100) cancelled` deb aniq korsatilgan.

4-savol

Kontekst: Dasturchi foydalanuvchi o'zgarishi mumkin bo'lgan ekranda quyidagicha yozdi:

```
LaunchedEffect(true) {
    api.sendHello(userId) // userId o'zgarishi mumkin
}
```

Vazifa: Nega LaunchedEffect(true) “while(true) kabi shubhali” deb atalgan? Qaysi holatda doimiy kalit (constant key) o'rinli (ma'ruzadagi splash misoli)? Yuqoridagi kodni tuzating.

> **Javob:**
> LaunchedEffect(true) 'while(true)' kabi shubhali deb atalgan chunki uning kaliti doimiy (true), yani userId ozgarsa ham effect qayta ishga tushmaydi va eski foydalanuvchi boyicha qolib ketadi.
> Doimiy kalit (true yoki Unit) faqat butun ekran hayoti davomida aniq bir marta ishlaydigan narsalarga (masalan Splash ekranda) togri keladi.
> **Tuzatilgan kod:**
>
> ```
> LaunchedEffect(userId) {
>     api.sendHello(userId)
> }
> ```

5-savol

Kontekst: Talaba tugma ichida snackbar ko'rsatish uchun onClick ga LaunchedEffect(Unit) { snackbarHostState.showSnackbar("Hi") } yozdi. Kod kompilyatsiya bo'lmayapti.

Vazifa: Nega LaunchedEffect callback ichidan chaqirib bo'lmaydi? rememberCoroutineScope nimani qaytaradi va uning umri nimaga bog'liq (composable Composition'ni tark etganda nima bo'ladi)? To'g'ri variantni yozing va showSnackbar suspend funksiya ekanining ahamiyatini tushuntiring. Qoida: "composable paydo bo'lgani uchun boshlanadigan ish" va "foydalanuvchi biror narsa qilgani uchun boshlanadigan ish" uchun qaysi API'lar tanlanadi?

> **Javob:**
> LaunchedEffect bu @Composable funksiya bolgani uchun uni oddiy callback (onClick) ichidan chaqirib bolmaydi, kompilyator xato beradi.
> rememberCoroutineScope ushbu composable umriga boglangan CoroutineScope ni qaytaradi. Ekran yopilsa shu scope dagi hamma ishlar bekor boladi.
> **To'g'ri variant:** `Button(onClick = { scope.launch { snackbarHostState.showSnackbar("Hi") } })`.
> showSnackbar suspend funksiya bolgani uchun uni korutina ichida chaqirish shart.
> **Qoida:** Composable paydo bolganda boshlanadigan ish uchun **LaunchedEffect**, foydalanuvchi tugma bosganda esa **rememberCoroutineScope** tanlanadi.

6-savol

Kontekst: LandingScreen(onTimeout) splash ekrani 2 soniya kutib, onTimeout() ni chaqiradi. Ota composable recompose bo'lganda unga yangi onTimeout lambda beradi:

```
@Composable
fun LandingScreen(onTimeout: () → Unit) {
    val currentOnTimeout by rememberUpdatedState(onTimeout)
    LaunchedEffect(true) {
        delay(SplashWaitTimeMillis)
        currentOnTimeout()
    }
}
```

Vazifa: Nega effect eski lambdani (captured) chaqirib yuborishi mumkin? rememberUpdatedState buni qanday hal qiladi? Nega onTimeout ni effect kaliti qilish noto'g'ri (kutish qaytadan boshlanadi)? Ma'ruzadagi model chiqishi (captured -> onTimeout v1, updatedState -> onTimeout v3) nimani isbotlaydi?

> **Javob:**
> LaunchedEffect(true) ichidagi delay kutayotgan paytda ota composable recompose bosa, effect eski lambdani (captured) ushlab qoladi va kechiksa ham eski versiyani chaqirib yuboradi.
> rememberUpdatedState har safar eng songgi berilgan lambdani eslab qoladi va effectni qayta boshlamasdan yangi lambdani chaqirishni taminlaydi.
> onTimeout ni kalit qilsak timer har recompositionda noldan qayta boshlanib 2 soniya hech qachon tugamasligi mumkin.
> Model chiqishi (captured -> v1, updatedState -> v3) rememberUpdatedState har doim eng oxirgi holatni ushlashini isbotlaydi.

7-savol

Kontekst: Ekran LifecycleEventObserver qo'shadi, lekin dasturchi uni olib tashlashni unutdi va onDispose yozmadi.

Vazifa: Nega tozalash (cleanup) kerak va nima uchun bu xotira oqishiga (leak) olib keladi? onDispose qoidasi qanday (qaysi o'rinda turishi kerak, aks holda nima bo'ladi)? Bo'sh onDispose nega yomon amaliyot va u o'rniga qaysi effect'lar mosroq bo'lishi mumkin? Kodni tuzating.

> **Javob:**
> Agar DisposableEffect da cleanup qilinmasa listener xotirada qolib ketadi va xotira oqishi (memory leak) yuzaga keladi, ekran yopilsa ham xotirada ishlab yotadi.
> onDispose qoidasi: u DisposableEffect blokining eng oxirgi qatorida turishi shart aks holda xato beradi.
> Bosh onDispose yozish yomon amaliyot chunki tozalaydigan narsa bomasa uning orniga LaunchedEffect yoki SideEffect ishlatish kerak.
> **Tuzatilgan kod:**

```
DisposableEffect(lifecycleOwner) {
    val observer = LifecycleEventObserver { _, event → /* ... */ }
    lifecycleOwner.lifecycle.addObserver(observer)
    onDispose {
        lifecycleOwner.lifecycle.removeObserver(observer)
    }
}
```

8-savol

Kontekst: Quyidagi MiniDisposable modeli DisposableEffect ning qoidalarini taqlid qiladi: kalit o'zgarsa, eski effect tozalanadi:

```
fun screen(owner: String) = comp.disposableEffect(owner) {
    println("$owner.addObserver(observer)")
    val onDispose = { println("$owner.removeObserver(observer)") }
    onDispose
}
screen("A"); screen("A") // bir xil owner
screen("B")              // owner o'zgardi
comp.leave()             // Composition'dan chiqdi
```

Vazifa: Dasturni ishga tushirmasdan konsol chiqishini to'liq (to'rt qator) yozing. Nega ikkinchi screen("A") hech narsa chiqarmaydi, va B kelganda nima birinchi bajariladi: eski observer olib tashlanishi yoki yangisi qo'shilishi? Natijani ishga tushirib tekshiring.

> **Javob:**
> **Konsol chiqishi (4 qator):**
> 1. `A.addObserver(observer)`
> 2. `A.removeObserver(observer)`
> 3. `B.addObserver(observer)`
> 4. `B.removeObserver(observer)`
> **Sababi:** Ikkinchi screen("A") da owner ozgarmagani uchun effect qayta ishlamaydi va hech narsa chiqarmaydi. B kelganda birinchi navbatda eski observer tozalanadi (`A.removeObserver`), keyin yangisi qoshiladi (`B.addObserver`).

9-savol

Kontekst: Analytics kutubxonasiga foydalanuvchi turi (userType) uzatilishi kerak. Dasturchi uni composable tanasida to'g'ridan-to'g'ri o'rnatadi.

Vazifa: SideEffect qachon ishga tushadi? To'g'ridan-to'g'ri chaqirish nega bekor qilingan (discard) composition urinishida ham kutubxonaga tegib ketishi mumkin? Ma'ruzadagi attempt modelining uch urinishida (direct call, commits=false; SideEffect, commits=false; SideEffect, commits=true) analytics qachon chaqirilishini tushuntiring. SideEffect ning kalitlari bormi?

> **Javob:**
> SideEffect faqat har bir muvaffaqiyatli recomposition tugab, natija ekranga commit qilingandan keyingina ishga tushadi.
> Togridan-togri chaqirish Compose ning bekor qilingan (discard) urinishlarida ham ishlab ketib kutubxonaga notogri log yuboradi.
> Attempt modelida: direct call commits=false da ham ishlab ketadi; SideEffect commits=false da ishlamaydi, faqat commits=true bolgandagina analytics ni chaqiradi.
> SideEffect ning kalitlari bolmaydi, u har bir muvaffaqiyatli recomposition dan keyin doimiy ishlaydi.

10-savol

Kontekst: Rasm yuklash holatlari: Loading, Success va Error. Dasturchi uchun loadNetworkImage(url) funksiyasini produceState bilan yozish so'raldi.

Vazifa: produceState nima qiladi va producer qachon ishga tushadi va bekor qilinadi? Qaytarilgan State nega “conflates” (bir xil qiymat berilsa recomposition bo'lmaydi), va ma'ruzadagi modelda bu qanday ko'rinadi? loadNetworkImage nega kichik harf bilan boshlanadi (qaytariladigan qiymati bor composable qoidasi)?

> **Javob:**
> produceState Compose tashqarisidagi suspend manbani Compose State ga aylantiradi. Composable ekranga chiqqanda producer boshlanadi, ekrandan ketganda avtomat bekor qilinadi.
> Qaytarilgan State qiymat ozgarmasa recomposition chaqirmaydi (conflates). Modelda bir xil qiymat takrorlanganda qayta chizilmasligi bilan korinadi.
> loadNetworkImage nomining kichik harfda yozilishi: Compose qoidasiga kora agar funksiya qiymat qaytarsa (`State<T>`) u kichik harf bilan boshlanadi.

11-savol

Kontekst: Jamoadosh produceState sehrli maxsus API deb hisoblaydi. Ma'ruzadagi Key Point esa uning “ichida boshqa effect'lar ishlatilgani”ni aytadi.

Vazifa: produceState ni remember { mutableStateOf(...) } va LaunchedEffect yordamida o'zingiz yozib chiqing (TODO o'rniga). Nega yaratilgan funksiya ham LaunchedEffect kabi xatti-harakat qiladi va kalit o'zgarganda nima bo'ladi?

> **Javob:**
> **produceState ning o'zimiz yozgan kodi:**
>
> ```
> @Composable
> fun <T> myProduceState(
>     initialValue: T,
>     key: Any?,
>     producer: suspend MutableState<T>.() → Unit
> ): State<T> {
>     val state = remember { mutableStateOf(initialValue) }
>     LaunchedEffect(key) {
>         state.producer()
>     }
>     return state
> }
> ```
>
> Ichida LaunchedEffect ishlatilgani sababli kalit ozgarganda eski producer bekor bolib yangisi boshlanadi.

12-savol

Kontekst: “Tepaga qaytish” tugmasi ro'yxatning birinchi ko'rinuvchi elementi har o'zgarganda qayta chiziladi, vaholanki tugma faqat indeks 0 dan katta bo'lgandagina kerak:

```
val listState = rememberLazyListState()
val showButton by remember {
    derivedStateOf { listState.firstVisibleItemIndex > 0 }
}
```

Vazifa: Nega to'g'ridan-to'g'ri o'qish keraksiz recomposition keltiradi va derivedStateOf buni qanday hal qiladi? Nega u remember ichiga o'ralgan? Ma'ruzadagi modelda indekslar [0, 0, 1, 2, 3, 4, 5, 3, 1, 0] bo'lganda to'g'ridan-to'g'ri o'qishda 9 ta, derivedStateOf bilan 3 ta recomposition chiqishining sababini hisoblab ko'rsating.

> **Javob:**
> Togridan-togri oqishda scroll ning har bir pikseli yoki indeksi ozgarganda keraksiz recomposition boladi. derivedStateOf esa faqat hisoblangan boolean natija ozgargandagina (false -> true yoki true -> false) recomposition chaqiradi.
> U remember ichiga o'ralgan bolishi shart aks holda har recompositionda yangitdan obekt yaratilib kesh ishlamay qoladi.
> **Hisoblash:** [0, 0, 1, 2, 3, 4, 5, 3, 1, 0] ro'yxatida togridan-togri oqishda 9 ta holat almashinuvi bolib 9 marta recompose boladi. derivedStateOf bilan esa: boshida 0 (false), keyin 1 da true boladi (+1), keyin 5 dan qaytib 0 bolganda false boladi (+1), jami 3 ta recomposition chiqadi.

13-savol

Kontekst: Talaba ikki holatni birlashtirib, “state hosil qilyapman” deb derivedStateOf ishlatdi:

```
var firstName by remember { mutableStateOf("") }
var lastName by remember { mutableStateOf("") }
val fullNameBad by remember {
    derivedStateOf { "$firstName $lastName" }
}
```

Vazifa: Nega bu holatda derivedStateOf ortiqcha xarajat? To'g'ri variantni yozing. derivedStateOf ni qo'llash mezoni nima (kirish va natija o'zgarish chastotalarini solishtiring)? Ma'ruzadagi “Caution” ogohlantirishini o'z so'zlaringiz bilan yozing.

> **Javob:**
> Bu yerda firstName va lastName ozgarish chastotasi bilan fullNameBad ozgarish chastotasi 1:1 ga teng (har harf yozilganda natija ham ozgaradi). Bunday holatda derivedStateOf ishlatish foydasiz va ortiqcha xotira xarajati.
> **To'g'ri variant:** `val fullName = "$firstName $lastName"`
> **derivedStateOf qo'llash mezoni:** Kirish qiymati juda tez ozgarib, chiquvchi natija kam ozgargandagina (chastota farqi katta bolganda) ishlatiladi.
> **Caution ogohlantirishi:** Agar natija kiruvchi state bilan bir xil tezlikda ozgaradigan bolsa derivedStateOf ni ishlatmang, u foyda bermaydi.

14-savol

Kontekst: Ma'ruzadagi “1 point question” — ovozli xabar waveform'i (to'lqin shakli) misolida derivedStateOf qachon ishlatilishi va qachon ishlatilmasligi so'raladi.

Vazifa: derivedStateOf ni qaysi shartda ishlatasiz? Qaysi uchta holatda ishlatmaysiz (amplitudalar oldindan hisoblangan; waveform statik; waveform LaunchedEffect yoki animate* bilan qo'lda yangilanadi)? Har bir holat uchun sababini yozing.

> **Javob:**
> derivedStateOf ni ovozli xabar vaqti (ms) juda tez yangilanib turganda, ekranda faqat qaysi to'lqin bar'i faollashishi kerakligini hisoblashda ishlatamiz.
> **Ishlatilmaydigan 3 ta holat:**
> 1. *Amplitudalar oldindan hisoblangan bolsa:* ma'lumot allaqachon tayyor, hisoblash keremas.
> 2. *Waveform statik bolsa:* holat umuman ozgarmaydi derivedStateOf ortiqcha.
> 3. *LaunchedEffect yoki animate* bilan qolda yangilansa:* animatsiya ozi recompositionni boshqaradi.

15-savol

Kontekst: Foydalanuvchi ro'yxatni birinchi elementdan o'tkazib yuborganda analytics'ga hodisa yuborish kerak:

```
LaunchedEffect(listState) {
    snapshotFlow { listState.firstVisibleItemIndex }
        .map { index → index > 0 }
        .distinctUntilChanged()
        .filter { it == true }
        .collect {
            MyAnalyticsService.sendScrolledPastFirstItemEvent()
        }
}
```

Vazifa: snapshotFlow nima qiladi va nega “cold Flow” deyiladi? map, distinctUntilChanged va filter operatorlari har biri nima qiladi? Nega collect LaunchedEffect ichida? Model oqimi 0, 0, 1, 2, 5, 3, 0, 0, 1, 4 bo'lganda nechta hodisa yuboriladi? Qadamma-qadam hisoblang.

> **Javob:**
> snapshotFlow Compose State ni Flow ga aylantiradi. U 'cold Flow' chunki collect qilinmaguncha hech narsa hisoblamaydi.
> map indeksni boolean qiladi, distinctUntilChanged bir xil takrorlangan qiymatlarni tashlab yuboradi, filter faqat true bolganini otkazadi.
> collect suspend funksiya bolgani uchun LaunchedEffect ichida korutina orqali chaqiriladi.
> **Qadamma-qadam hisob:** 0 (false), 0 (tashlanadi), 1 (true -> 1-hodisa), 2 (true -> distinct tashlaydi), 5 (true -> tashlaydi), 3 (true -> tashlaydi), 0 (false), 0 (tashlanadi), 1 (true -> 2-hodisa), 4 (true -> tashlaydi). Jami **2 ta** hodisa yuboriladi.

16-savol

Kontekst: Ma'ruzadagi Effect modeli uchta variantda tick 1 dan 4 gacha o'zgaradi va owner faqat 3-tickda "A" dan "B" ga o'zgaradi.

Vazifa: Jadvalni to'ldiring. Qaysi variant xato (eskirgan qiymat bilan ishlaydi), qaysi biri to'g'ri va qaysi biri samarasiz? Natijani ma'ruzadagi modelni ishga tushirib tekshiring.

Kalitlar	Restart soni (bashorat)	Sababi
`keys = []`	0 marta (hech qachon restart bo'lmaydi)	**XATO:** owner o'zgarsa ham yangilanmaydi, eskirgan ma'lumot bilan ishlaydi.
`keys = [owner]`	1 marta (faqat 3-tickda owner 'B' ga o'zgarganda)	**TO'G'RI:** faqat bog'liq obyekt o'zgargandagina effect toza qayta boshlanadi.
`keys = [owner, tick]`	4 marta (har bir tickda qayta ishga tushadi)	**SAMARASIZ:** har bir tickda keraksiz tozalash va qayta yuklash bo'ladi.

17-savol

Kontekst: HomeScreen(lifecycleOwner, onStart, onStop) ichida onStart va onStop rememberUpdatedState bilan o'ralgan, DisposableEffect esa lifecycle observer qo'shadi.

Vazifa: Qaysi o'zgaruvchilar effect kaliti bo'lishi kerak va qaysilari kerak emas? Nega currentOnStart va currentOnStop kalit emas? lifecycleOwner kalit qilinmasa va u o'zgarsa, qanday muammo yuz beradi? Ma'ruzadagi "Key Point" qoidasini yozing.

> **Javob:**
> Faqat lifecycleOwner effect kaliti bolishi kerak. currentOnStart va currentOnStop kalit bolmasligi shart chunki ular rememberUpdatedState bilan olingan, ularning yangilanishi effectni qayta boshlamasligi kerak.
> Agar lifecycleOwner kalit qilinmasa va boshqa activity/ekranga otilsa eski lifecycle kuzatilib qoladi va xotira oqadi.
> **Key Point qoidasi:** Effect tanasida ishlatiladigan barcha dinamik parametrlar kalit bolishi shart, faqat effectni qayta ishga tushirmasligi kerak bolgan parametrlar rememberUpdatedState bilan oralsagina kalitdan chiqariladi.

18-savol

Kontekst: Code review'da quyidagi composable kelib tushdi. Unda to'rtta effect bilan bog'liq xato bor:

```
@Composable
fun ChatScreen(userId: String, onClose: () → Unit) {
    val scope = rememberCoroutineScope()
    val owner = LocalLifecycleOwner.current
    val observer = remember { LifecycleEventObserver { _, _ → } }
    viewModel.markOpened(userId)                    // (1)
    LaunchedEffect(true) { delay(5_000); onClose() } // (2)
    DisposableEffect(owner) {                       // (3)
        owner.lifecycle.addObserver(observer)
    }
    Button(onClick = {                              // (4)
        LaunchedEffect(Unit) { send() }
    }) { Text("Send") }
}
```

Vazifa: To'rtta xatoni toping, har birining sababini yozing va tuzatilgan variantni taqdim eting.

Javob:
1) `viewModel.markOpened(userId)` composable tanasida turibdi -> har recompositionda qayta chaqiriladi. Buni `LaunchedEffect(userId)` ichiga olish kerak.
2) `LaunchedEffect(true)` doimiy kalitda yozilgan -> userId ozgarsa timer qayta ishlamaydi va onClose eski lambdani ushlab qoladi. `rememberUpdatedState(onClose)` qilinishi shart.
3) `DisposableEffect(owner)` ichida onDispose yoq -> kompilyatsiya bermaydi va observer xotirada qolib ketadi. `onDispose { owner.lifecycle.removeObserver(observer) }` yozilishi shart.
4) `Button(onClick = { LaunchedEffect(Unit) { send() } })` -> callback ichida composable chaqirib bolmaydi. rememberCoroutineScope olib, `scope.launch { send() }` yozilishi shart.
Tuzatilgan kod:

```
@Composable
fun ChatScreen(userId: String, onClose: () → Unit) {
    val scope = rememberCoroutineScope()
    val owner = LocalLifecycleOwner.current
    val currentOnClose by rememberUpdatedState(onClose)
    val observer = remember { LifecycleEventObserver { _, _ → } }

    LaunchedEffect(userId) {
        viewModel.markOpened(userId)
        delay(5_000)
        currentOnClose()
    }
    DisposableEffect(owner) {
        owner.lifecycle.addObserver(observer)
        onDispose { owner.lifecycle.removeObserver(observer) }
    }
    Button(onClick = { scope.launch { send() } }) { Text("Send") }
}
```

19-savol

Kontekst: Ma'ruzaning yakuniy jadvali sakkizta API'ni uchta savol bo'yicha tartibga soladi: qachon ishlaydi, nima tozalanadi, nima restart qiladi.

Vazifa: Jadvalni mustaqil to'ldiring. Keyin o'zingizning loyihangizdan har bir API uchun bittadan real foydalanish misolini o'ylab toping.

API	Qachon ishlaydi	Tozalash	Restart sababi
LaunchedEffect	Composition'ga kirganda yoki kalit o'zgarganda	Korutina bekor qilinadi (cancel)	Kalit (key) o'zgarganda
rememberCoroutineScope	UI callback (onClick) ichida chaqirilganda	Composition'dan chiqqanda bekor bo'ladi	Restart bo'lmaydi
rememberUpdatedState	Har bir recompositionda yangi qiymatni ushlashda	Tozalash shart emas	Restart bo'lmaydi
DisposableEffect	Composition'ga kirganda yoki kalit o'zgarganda	onDispose bloki bajariladi	Kalit (key) o'zgarganda
SideEffect	Har bir muvaffaqiyatli recomposition / commit'dan keyin	Tozalash yo'q	Har doim (har recompositionda)
produceState	Composition'ga kirganda yoki kalit o'zgarganda	Producer korutinasi bekor qilinadi	Kalit (key) o'zgarganda
derivedStateOf	Kuzatilayotgan kiruvchi state'lar o'zgarganda	Tozalash yo'q	Natija o'zgarganda
snapshotFlow	LaunchedEffect ichida flow boshlanganda	Flow oqimi bekor bo'ladi	Compose state o'zgarganda

B qism. Amaliy topshiriqlar haqida hisobot

Barcha 6 ta amaliy topshiriq `uz.ttpu.sideeffectslab` paketida Android Studio loyihasi sifatida to'liq bajarildi va alohida semantik commitlar bilan taqdim etildi:

- **1-topshiriq:** `LandingScreen.kt` — Splash ekrani, `LaunchedEffect` va `rememberUpdatedState` orqali eskirib qolmaydigan dinamik lambda chaqirish sinovi.
- **2-topshiriq:** `SnackbarScreen.kt` — Scaffold + SnackbarHost + `rememberCoroutineScope` orqali tugma bosilganda snackbar chiqarish.
- **3-topshiriq:** `LifecycleScreen.kt` — `DisposableEffect` bilan `LifecycleEventObserver` qo'shish va onDispose da xotirani tozalash.
- **4-topshiriq:** `AnalyticsScreen.kt` — Compose state ni tashqi `Analytics` obyektiga `SideEffect` orqali xavfsiz uzatish va recomposition sonini sanash.
- **5-topshiriq:** `ScrollScreen.kt` — 100 elementli LazyColumn, `derivedStateOf` bilan recomposition sonini keskin tejash va `snapshotFlow` bilan yagona analytics hodisasi yuborish.
- **6-topshiriq (Bonus):** `ImageScreen.kt` — Soxta repozitoriy va `produceState` yordamida Loading, Success va Error holatlarini boshqarish.